In [14]:
```
import pandas as pd
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
```

In [15]:
```
train_data = pd.read_csv("fashion-mnist_train.csv")
test_data = pd.read_csv("fashion-mnist_test.csv")

print(train_data.shape)
print(test_data.shape)
```

```
(60000, 785)
(10000, 785)
```

In [16]:
```
# Separate labels from pixel values
y_train = train_data.iloc[:, 0].values
X_train = train_data.iloc[:, 1:].values

y_test = test_data.iloc[:, 0].values
X_test = test_data.iloc[:, 1:].values

print(X_train.shape, y_train.shape)
print(X_test.shape, y_test.shape)
```

```
(60000, 784) (60000,)
(10000, 784) (10000,)
```

In [17]:
```
X_train = X_train / 255.0
X_test = X_test / 255.0
```

In [18]:
```
model = Sequential([
    Dense(128, activation="relu", input_shape=(784,)),
    Dense(10, activation="softmax")
])

model.summary()
```

```
C:\Users\ombah\AppData\Roaming\Python\Python313\site-packages\keras\src\layers\core\dense.py:107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 10)	1,290

Total params: 101,770 (397.54 KB)

Trainable params: 101,770 (397.54 KB)

Non-trainable params: 0 (0.00 B)

In [19]:
```
model.compile(
    optimizer="sgd",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
```

In [20]:
```
history = model.fit(
    X_train,
    y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.1
)
```

```
Epoch 1/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.7454 - loss: 0.7852 - val_accuracy: 0.7990 - val_loss: 0.6062
Epoch 2/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.8201 - loss: 0.5311 - val_accuracy: 0.8263 - val_loss: 0.5228
Epoch 3/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.8345 - loss: 0.4818 - val_accuracy: 0.8333 - val_loss: 0.4903
Epoch 4/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 3s 1ms/step - accuracy: 0.8424 - loss: 0.4537 - val_accuracy: 0.8377 - val_loss: 0.4684
Epoch 5/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.8483 - loss: 0.4353 - val_accuracy: 0.8390 - val_loss: 0.4646
Epoch 6/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.8538 - loss: 0.4201 - val_accuracy: 0.8440 - val_loss: 0.4519
Epoch 7/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.8579 - loss: 0.4081 - val_accuracy: 0.8535 - val_loss: 0.4378
Epoch 8/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.8614 - loss: 0.3980 - val_accuracy: 0.8558 - val_loss: 0.4180
Epoch 9/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.8646 - loss: 0.3882 - val_accuracy: 0.8523 - val_loss: 0.4310
Epoch 10/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.8670 - loss: 0.3810 - val_accuracy: 0.8568 - val_loss: 0.4224
```

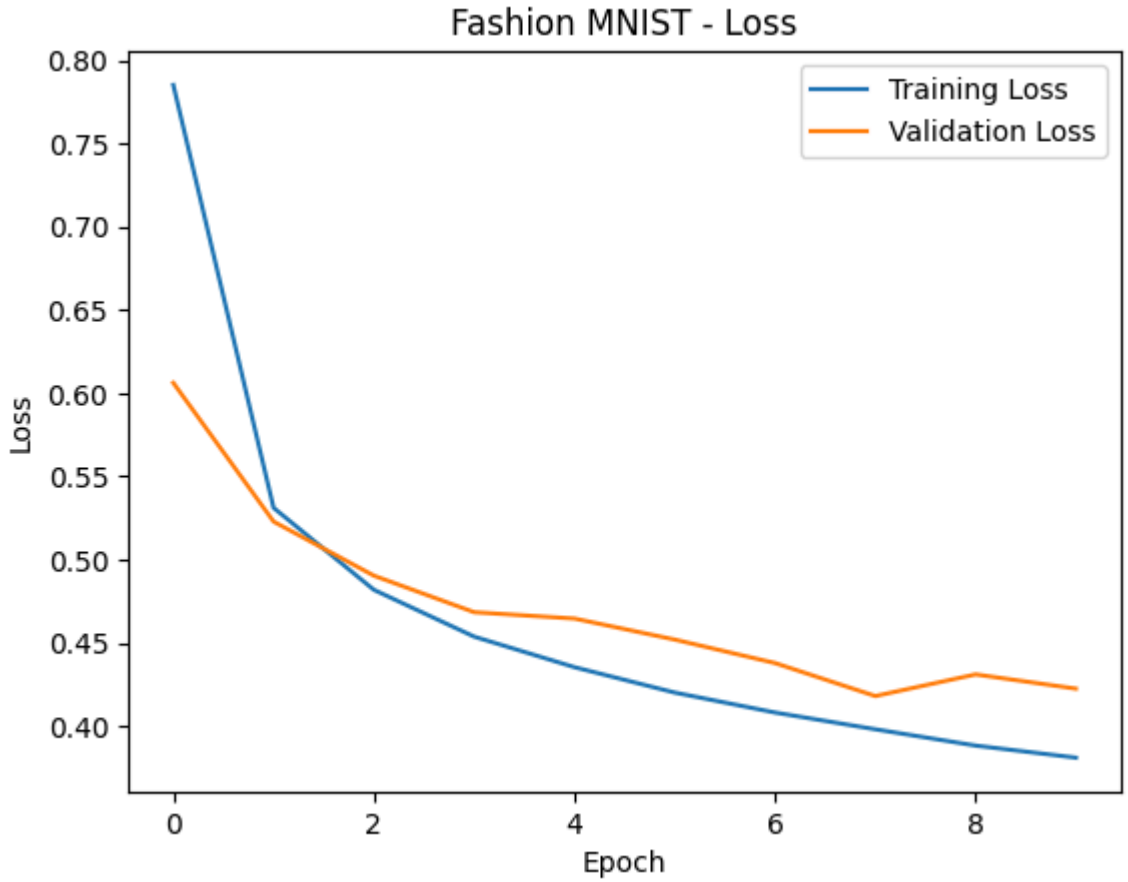
In [21]:
```python
test_loss, test_accuracy = model.evaluate(X_test, y_test, verbose=0)

print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)
```

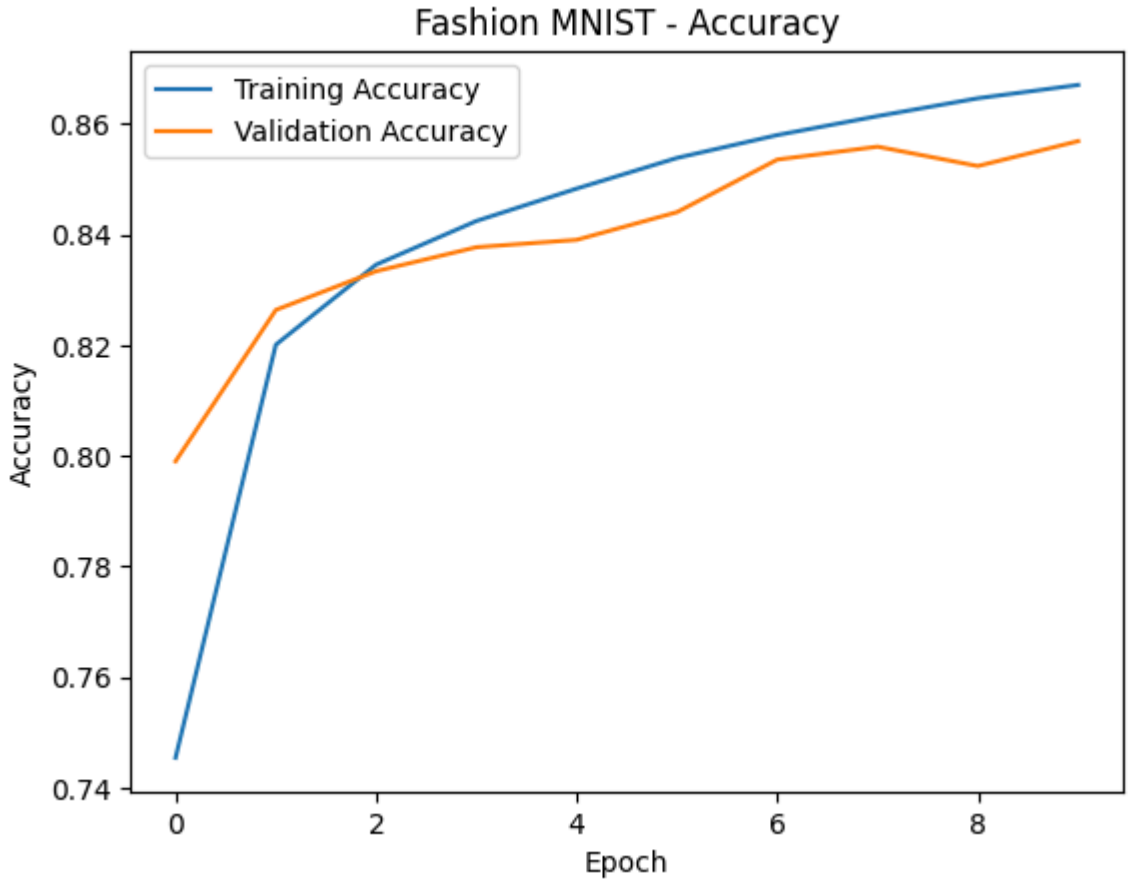
```
Test Loss: 0.40121182799339294
Test Accuracy: 0.8600000143051147
```

In [22]:
```python
import matplotlib.pyplot as plt

plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.title("Fashion MNIST - Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.show()
```



In [23]:
```
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.title("Fashion MNIST - Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```


Fashion MNIST - Accuracy
Training Accuracy
Validation Accuracy
Accuracy
Epoch
0.74
0.76
0.78
0.80
0.82
0.84
0.86
0
2
4
6
8

In [ ]: